

Система автоматизированного обогащения товарных данных для интернет-магазина

Фролов М. А.

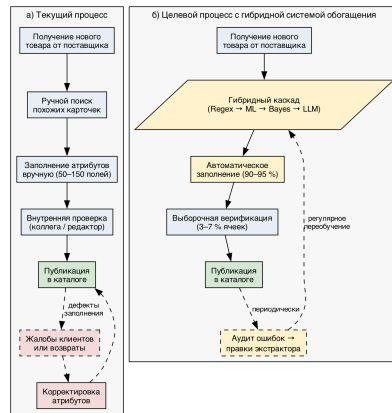
Московский авиационный институт
Институт № 8, кафедра 806 «Вычислительная математика и программирование»
Научный руководитель: Судаков В. А.

Москва, 2026

Проблемные вопросы практики обогащения карточек

- ▶ Каталоги маркетплейсов — **десятки миллионов SKU**.
- ▶ Партнёры присылают **лишь базовый набор** полей.
- ▶ PIM-системы **хранят**, но **не извлекают** атрибуты.
- ▶ Ручное заполнение — **узкое место** вывода ассортимента.

$\sim 10^7$ SKU · часы на карточку ·
линейная стоимость от объёма



Актуальность направления и задача ВКР

Противоречия

- ▶ **Процесс** — $10^7 + \text{SKU}$ vs пропускная способность контент-менеджеров.
- ▶ **Средства** — PIM хранит, но не извлекает атрибуты.
- ▶ **Технология** — точность БЯМ vs её **цена** на промышленных объёмах.

Задача ВКР. Программное средство обогащения товарных данных с заданной точностью при **кратном сокращении стоимости** относительно прямого вызова большой языковой модели и допускающее локальное развёртывание.

*⇒ перенос акцента с «одна большая модель на всё» на
поатрибутную маршрутизацию между слоями разной стоимости.*

Аналоги: пробел в существующих подходах

Критерий	TXtract	MAVE	FrugalGPT	AutoMix	Наша система
Per-attribute policy	✓	✓	—	—	✓
Калиброванный отказ от ответа	—	—	—	✓	✓
Cost-aware routing	—	—	✓	✓	✓
Code-disjoint eval	—	—	—	—	✓
Гибридная архитектура	—	—	—	—	✓
Локальное развёртывание	частично	частично	—	—	✓

Ни одна из существующих систем не объединяет **per-attr policy**, **code-disjoint eval** и **гибридную архитектуру** одновременно.

TXtract / MAVE — реализации Amazon / Google: возможен self-hosted, но требует GPU и инфраструктуры BERT-семейства.

Цель и задачи ВКР

Цель. Программное средство обогащения товарных данных на основе гибридной каскадной архитектуры.

Частные задачи

1. Разработать архитектуру гибридного каскадного конвейера (5 слоёв) с поатрибутной политикой маршрутизации.
2. Реализовать каждый слой как самостоятельный программный компонент с независимым обновлением.
3. Построить эталонную разметку и тестовую выборку без пересечения товарных кодов (code-disjoint).
4. Провести экспериментальную оценку: точность, покрытие, стоимость; сравнение с прямым вызовом БЯМ.
5. Разработать демонстрационный программный комплекс с REST API и веб-интерфейсом.

Объект, предмет и рабочее предположение

Объект. Процессы обогащения карточек товаров в интернет-магазинах.

Предмет. Методы и алгоритмы извлечения атрибутов, политики маршрутизации между слоями разной стоимости и протоколы их оценки.

Рабочее предположение.

- ▶ **Каскад** из слоёв разной стоимости (regex → ML → Bayes → LLM) обеспечивает заданную точность обогащения товарных данных.
- ▶ **Поатрибутная политика эскалации** по калиброванной уверенностикратно сокращает стоимость относительно прямого вызова большой языковой модели.
- ▶ **Локальное развёртывание** возможно на коммерчески доступной инфраструктуре.

Исходные посылки

1. Унаследованная инфраструктура

PIM (Akeneo, Pimcore, Salsify) хранит, но не извлекает атрибуты.

2. Теоретическая основа

Многоязычные эмбединги, калиброванная классификация (Platt + isotonic), байесовские сети, prompt-инженерия (few-shot, schema-constrained).

3. Технологический стек

Python / FastAPI / Go-шлюз / React; Docker-compose; XGBoost, sentence-transformers, pgmpy; Ollama для локальной языковой модели.

4. Открытые данные

Open Food Facts (4,5 млн SKU, ~40 % FR / 10 % EN / 7 % DE) + открытые источники по электронике для холодного старта.

Методы исследования

Системный анализ

Декомпозиция процесса, выявление противоречий.

Управление требованиями

ФР / НФР / А-требования, интегральный критерий Q.

Машинное обучение

Эмбединги + XGBoost, калибровка Platt + isotonic.

Промпт-инженерия

Few-shot со схемой ответа, селективная эскалация.

Предрегистрируемый эксперимент

Непересекающиеся бренды, McNemar + Бонферрони, Wilson 95 % CI.

Итеративная разработка

Цикл

аудит → правки → переобучение → измерение.

Основные результаты работы

1. **Требования и качество:** 6 ФР, 5 НФР, 3 А; критерий Q с 4 показателями.
2. **Методика:** 4-слойный каскад с поатрибутной политикой эскалации.
3. **Архитектура:** REST-микросервис, упакованный в `docker-compose`.
4. **Реализация:** ~15 тыс. строк Python, 21 пара «категория-атрибут» (8/6/7 для `pasta/chocolate/cheeses`), работающий e2e демо.
5. **Оценка:** достигнуты целевые показатели качества (точность, сокращение стоимости, покрытие, калибровка) — детали далее.

Формальная постановка задачи

Дано. Товар как кортеж

$x = (\text{name}, \text{brand}, \text{ingr}, \text{qty}, \text{code}) \in \mathcal{X}$.

Множество целевых атрибутов $A = \{a_1, \dots, a_K\}$,

$K = 21$ (паста 8, шоколад 7, сыры 6). Для каждого

$a \in A$ — конечный словарь $\mathcal{Y}_a$ с отказом:

$\mathcal{Y}_a^\emptyset = \mathcal{Y}_a \cup \{\perp\}$.

Найти. Политика обогащения как композиция четырёх слоёв

$$\pi_a = f_4^{(a)} \circ \sigma_3 \circ f_3^{(a)} \circ \sigma_2 \circ f_2^{(a)} \circ \sigma_1 \circ f_1^{(a)},$$

$f_i^{(a)} : \mathcal{X} \rightarrow \mathcal{Y}_a^\emptyset \times [0, 1]$ возвращает $(\hat{y}, c)$; σ_i — селективные ворота: принять $\hat{y}$ при $c \geq \tau_{i,a}$, иначе $\perp$ (переход к слою $i+1$).

Целевая функция. На размеченной выборке $\{(x_n, y_n^a)\}_{n=1}^N$ для каждого $a \in A$:

$$\text{acc}_a^{\text{e2e}}(\pi_a) = \frac{1}{N} \sum_n \mathbb{I}[\pi_a(x_n) = y_n^a],$$

$$\text{cost}_{\text{LLM}}(\pi) = \frac{1}{N|A|} \sum_{a,n} \mathbb{I}[\pi_a(x_n) \rightarrow f_4].$$

Оптимизационная задача.

$$\max_{\{\tau_{i,a}\}} \frac{1}{|A|} \sum_a \text{acc}_a^{\text{e2e}}(\pi_a) \text{ при } \text{cost}_{\text{LLM}}(\pi) \leq \beta.$$

Ограничения.

- $\beta = 10\%$ — бюджет вызовов запасного слоя;
- признаки только из x (анти-leakage: без `categories_tags`);
- оценка *code-disjoint*: $\text{code}(x_{\text{tr}}) \cap \text{code}(x_{\text{te}}) = \emptyset$;
- локальное развёртывание слоя 4.

Состав требуемых функций программного средства

Функциональные требования

- ▶ **F1.** Предсказание значения каждого целевого атрибута.
- ▶ **F2.** Возврат *слоя-источника* ($L_1 / L_2 / L_3 / L_4$).
- ▶ **F3.** Возврат *калиброванной уверенности* $c \in [0, 1]$.
- ▶ **F4.** Селективная эскалация на запасной слой языковой модели.
- ▶ **F5.** Байесовская валидация: флаг подозрительных решений.
- ▶ **F6.** Поатрибутная настройка через конфигурацию.

Нефункциональные требования

- ▶ **N1.** Доля вызовов языковой модели $\leq 10\%$ решений.
- ▶ **N2.** **Локальное развёртывание** (без внешних API).
- ▶ **N3.** Обобщение на непересекающихся товарных кодах (code-disjoint).
- ▶ **N4.** Воспроизводимость: `предрегистрация + reproduce.sh`.

Архитектурные требования. **A1.** Контейнеризация (docker-compose). **A2.** Стабильный REST-контракт между слоями. **A3.** Холодный старт: новая категория без правки ядра.

Система качества продукта и интегральный критерий

Качество описано **четырьмя показателями** (точность, стоимость, покрытие, калибровка), сведёнными в интегральный критерий Q:

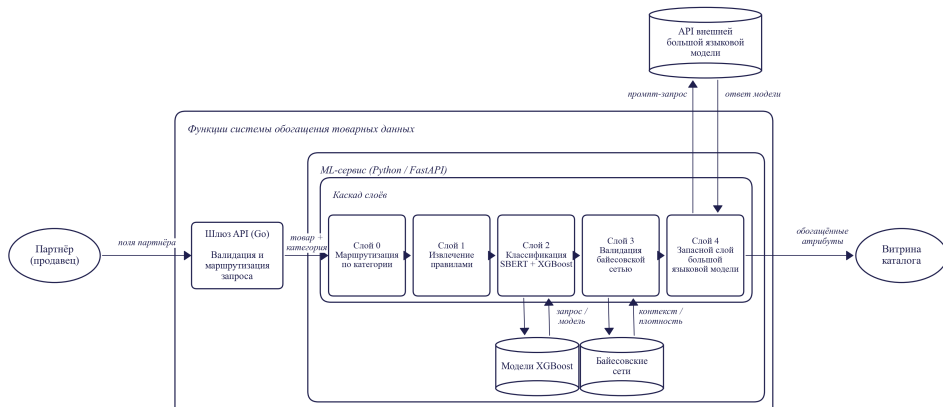
$$Q = w_1 \cdot \text{Acc} + w_2 \cdot \left(1 - \frac{C_{\text{LLM}}}{C_{\text{max}}}\right) + w_3 \cdot \text{Cov} - w_4 \cdot \text{ECE}$$

Acc — точность на непересекающихся товарных кодах (code-disjoint); $C_{\text{LLM}}/C_{\text{max}}$ — доля LLM-вызовов; Cov — покрытие каскадом до LLM; ECE — ошибка калибровки.

Веса: $w_1 = 0,40$, $w_2 = 0,30$, $w_3 = 0,20$, $w_4 = 0,10$ (приоритет точности над стоимостью согласован с типовыми требованиями РИМ-индустрии).

Показатель	Целевое	Достигнуто
Точность каскада	$\geq 90\%$	94,8 %
$C_{\text{LLM}}/C_{\text{max}}$	$\leq 0,10$	0,071
Cov (покрытие до LLM)	$\geq 0,90$	0,929
ECE (калибровка)	$\leq 0,05$	per-attr ниже целевого

Функциональная модель системы

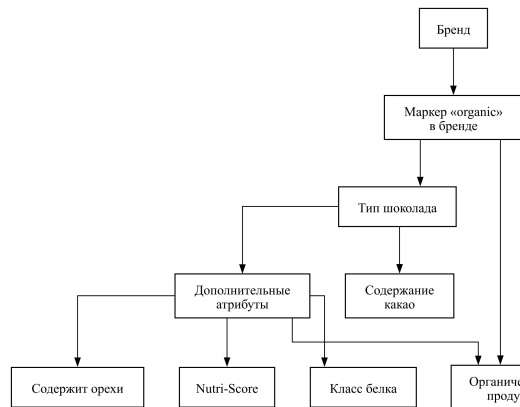


Слой 3: байесовский валидатор и DAG атрибутов

Слой 3 — pgmpy 1.1.2, структура направленного ациклического графа найдена алгоритмом **Hill Climb** с критерием **BIC**; условные вероятности через `BayesianEstimator` с априором `BDeu`.

Роль в production — селективный валидатор плотностей, а не классификатор:

- ▶ самостоятельная классификация $\approx 52\%$ (ниже majority baseline),
- ▶ но $P(\text{value} \mid \text{evidence})$ на нижнем 5-перцентиле даёт сигнал отбраковки,
- ▶ включён точно на **1 паре из 20** — `chocolate/contains_nuts`: **точность категории +6,6 пп** при росте затрат на LLM **+5,3 пп**



Ключевые алгоритмы каскада

Бюджет $\beta = 10\%$ (фактически 7,1 %); **21 классификатор** по парам (8/6/7 — pasta/choc/cheeses); сквозная точность каскада **94,8 %**, макро-F1 0,899.

Слой	Вход	Алгоритм	Условие эскалации	Реш.	Передаёт дальше
L0 router	name + brand + ingr	XGBoost на эмбединге + OOD-флаг	SBERT-category OOD-AUROC $\geq 0,80$; OOD $\Rightarrow$ слой 4 «как есть»	—	category $\rightarrow$ выбор π_{cat}
L1 regex	name + qty (числовые поля)	регулярные выражения с поатрибутной точностью 100 % на gold	нет матча $\Rightarrow$ слой 2	18,4 %	$(\hat{y}, c=1)$ при матче
L2 ML	SBERT мерный (4 partner-поля) 768-вектор	XGBoost (21 моделей), Platt + isotonic калибровка	$p_{max}(x) < \tau_a$ ($\tau_a \in [0,5; 0,9]$ из ECE) $\Rightarrow$ слой 3	73,8 %	$(\hat{y}, c)$ или $\perp$
L3 Bayes	brand + ML-предсказания других атрибутов	pgmpy 1.1.2: HillClimb+BIC, Variable Elimination	$P_B(\hat{y} \mid e) < \theta_a$ (5-й перцентиль обуч.) $\Rightarrow$ слой 4	0,7 %	флаг и/или $(\hat{y}, c)$
L4 LLM	все поля + few-shot JSON-схема отве-	gpt-oss-120b (open) / Sonnet 4.5 / Gemini 2.5 Flash	нет (последний слой)	7,1 %	$(\hat{y}_{LLM}, c)$

Данные и протокол оценки

Источник: Open Food Facts, 4,5 млн SKU (~40 % FR / 10 % EN / 7 % DE).

3 категории: паста (8 атр.), шоколад (7), сыры (7) — **21 пара** «категория-атрибут» (8/6/7).

Эталон: два уровня контрольной разметки — основной ($n = 3257$) и проверочный ($n = 615$, нижняя оценка точности).

Протокол: проверка на непересекающихся товарных кодах (code-disjoint).

Предрегистрация гипотезы о маршрутизаторе: cd9ac7a (2026-05-13).

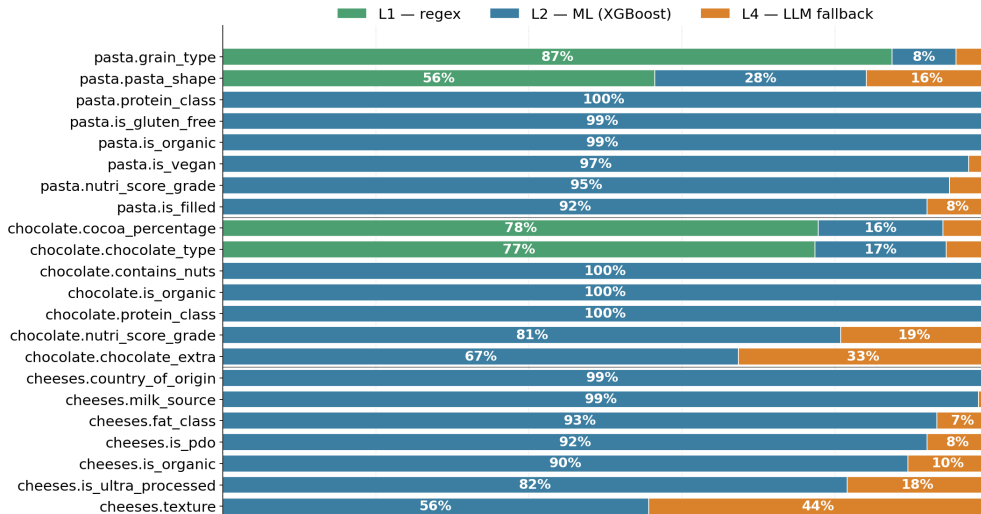
Категория	Атрибутов
Pasta	8
Chocolate	7
Cheeses	7
<i>Расширения:</i>	
Beverages	8
Cereals	8
Cosmetics	6
<i>Холодный старт:</i>	
Electronics	8

Главный результат: точность и стоимость

Метрика	Значение	<i>n</i>
Точность каскада (микро)	94,8 %	3257
Макро-F1 каскада	0,899	3257
Точность маршрутизатора категории	95,4 %	570
Покрытие сквозной цепочки	96,0 %	3257
Точность производственной цепочки	91,1 %	3257
Нижняя оценка (проверочная выборка)	87,5 %	615

Проверка на непересекающихся товарных кодах (code-disjoint). **Доля языковой модели: 7,1 %** ячеек $\Rightarrow$ сокращение стоимости в **14×** раз.

За счёт чего: вклад слоёв по 22 атрибутам



Результаты разработки программного средства

1. **Соответствие требованиям.** Все 6 ФР, 5 НФР, 3 А реализованы; **4/4** показателя качества достигли целевых.
2. **Объём.** ~15 тыс. строк Python; 35+ модулей; 7 категориальных схем; 21 пара «категория-атрибут» (8/6/7).
3. **Демо.** `docker-compose`: React + Go + FastAPI; 5 REST-эндпоинтов; ~200 мс на запрос.
4. **Воспроизводимость.** ~70 тестов; ноутбук как исполняемый §3.3; `reproduce.sh` восстанавливает headline-цифры.

Научная и практическая значимость

Научная значимость.

- ▶ Методика построения гибридного каскада с **поатрибутной калиброванной уверенностью** и **предрегистрируемым** протоколом оценки.
- ▶ Предрегистрированный **отказ** от гипотезы об обучаемом маршрутизаторе в пользу статической политики (простота и предсказуемость).
- ▶ Воспроизводимость методики подтверждена на новой категории (электроника) в режиме холодного старта.

Практическая значимость.

- ▶ Работающее ПО в `docker-compose`, интегрируется рядом с PIM.
- ▶ Кратное сокращение стоимости и ручной нагрузки контент-менеджеров.
- ▶ Возможность локального развёртывания — независимость от внешних API.

Достоверность и обоснованность результатов

Достоверность:

- ▶ проверка на непересекающихся товарных кодах (code-disjoint): основная выборка $n=3257$ + проверочная $n=615$ (нижняя оценка);
- ▶ **предрегистрация** протокола гипотезы о маршрутизаторе в git cd9ac7a до создания артефакта;
- ▶ Wilson 95 % CI на всех headline-числах;
- ▶ McNemar + поправка Бонферрони ($\alpha/3$);
- ▶ воспроизведение через `reproduce.sh`.

Обоснованность. Признанные методы: sentence-transformers, XGBoost, байесовские сети, калибровка Platt + isotonic; сравнение с TXtract, MAVE, OpenTag, FrugalGPT, AutoMix.

Реализация. `docker-compose` stack с REST API и Web-UI; код и `reproduce.sh`

Демонстрационный комплекс — экранные формы

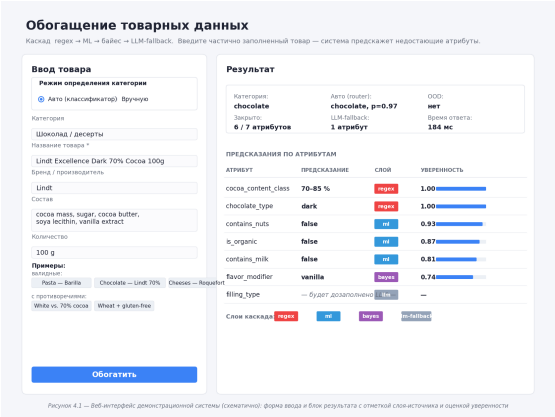


Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с меткой слов-источника и оценкой уверенности

Рекомендации по внедрению, эксплуатации и развитию

Внедрение

- ▶ `docker-compose` up рядом с PIM.
- ▶ Стартовый серебряный эталон из истории каталога.
- ▶ Поатрибутные пороги через JSON-конфиги.
- ▶ Языковая модель: внешний API или локальный Ollama.

Эксплуатация

- ▶ Мониторинг доли языковой модели ($\leq 10\%$).
- ▶ Точность на выборке $n \geq 200$ еженедельно.
- ▶ Аудит при росте байесовских флагов $> 5\%$.
- ▶ Верификация только отмеченных ячеек.

Развитие

- ▶ Холодный старт на новые категории.
- ▶ Расширение схем атрибутов.
- ▶ Замена базовой модели эмбедингов на более крупную.
- ▶ Подключение более дешёвых языковых моделей.

Оперативный эффект от внедрения

94,8 %

точность каскада
($n=3257$)

91,1 %

точность
производственной
цепочки

7,1 %

доля ячеек,
уходящих в языковую
модель

14×

сокращение стоимости
относительно прямого
вызова

⇒ кратное сокращение ручной нагрузки и стоимости обогащения
при сохранении локального развёртывания.

Заключение: задачи и полученные результаты

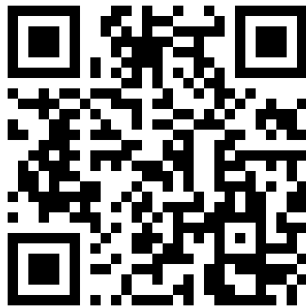
Задача	Результат
1. Архитектура каскада	5 слоёв с конъюнктивным AND-условием на пороги; явный отказ слоя; идентификация слоя-источника каждого предсказания.
2. Реализация слоёв	21 классификатор по 3 категориям (8/6/7); независимое обновление; <code>docker-compose</code> : Go-шлюз + Python/FastAPI ML-сервис + Web-UI.
3. Эталонная разметка	LLM-consensus gold $n=3257$ + human gold $n=615$; code-disjoint split (brand overlap 82–85 %).
4. Экспериментальная оценка	точность каскада 94,8 % / 0,899 макро-F1, производственная 91,1 %, человеческий gold 87,5 %; сокращение стоимости 14× архитектурное / 333× комбинированное.
5. Демо-комплекс	<code>docker-compose</code> (Go + FastAPI + Web-UI); REST-контракт; 3

Спасибо за внимание

Точность каскада **94,8 %**,
производственной цепочки **91,1 %**,
сокращение стоимости в **14×** раз.

Исходный код, ноутбук §3.3 в исполняемом
виде, демо-комплекс и текст работы — в
репозитории.

Вопросы?



github.com/Qworl/diploma

В0. Метрики и обозначения (для вопросов)

AND-условие слоя 2. Предсказание $\hat{y}$ принимается

$\iff P_{\text{ML}}(\hat{y} \mid x) \geq \tau_a \wedge P_B(\hat{y} \mid e) \geq \theta_a$ — формализация ансамблевой оценки уверенности как конъюнкции порогов ML-классификатора (τ_a) и байесовского валидатора (θ_a).

Метрики:

- ▶ $\text{acc}_k^{\text{cov}} = \frac{|\hat{y}_k = y_k \wedge \hat{y}_k \neq \text{null}|}{|\hat{y}_k \neq \text{null}|}$ — точность на покрытой части (только по ячейкам, где $\hat{y} \neq \perp$);
- ▶ $\text{cov}_k = \frac{|\hat{y}_k \neq \text{null}|}{N}$ — покрытие, доля ячеек с ненулевым предсказанием;
- ▶ $\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k$ — сквозная точность (отказ $\perp$ засчитывается как ошибка), production-realistic;
- ▶ cost_{LLM} — ожидаемая доля вызовов запасного слоя на 1000 ячеек (бюджет $\beta = 10\%$).

Связь $\text{acc}^{\text{e2e}} = \text{acc}^{\text{cov}} \cdot \text{cov}$ объясняет разрыв между только-каскадной точностью

В1. Отрицательный результат: обучаемый маршрутизатор

Гипотеза Н1: обучаемый

стоимостно-чувствительный маршрутизатор превосходит статическое правило хотя бы на одном из трёх бюджетов языковой модели (25 % / 40 % / 50 %).

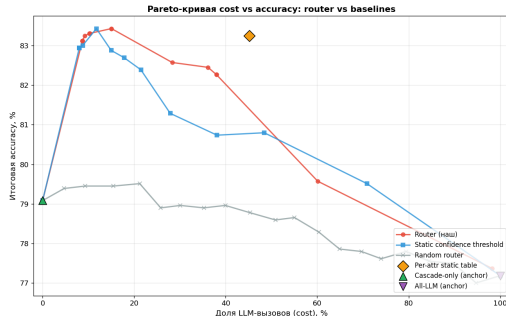
Предрегистрация: git cd9ac7a, 2026-05-13 02:11 (за 1 ч 11 мин до создания артефакта).

Поправка Бонферрони: $\alpha/3 \approx 0,0167$.

Минимально различимый эффект (MDE) при $n = 1539$: $\approx 4,4$ пп (парный McNemar).

Результат: $p_{\text{McNemar}} = 0,74/0,49/1,00$ для бюджетов 25 / 40 / 50 % (все $> \alpha/3$).

Парето: обучаемый vs статический

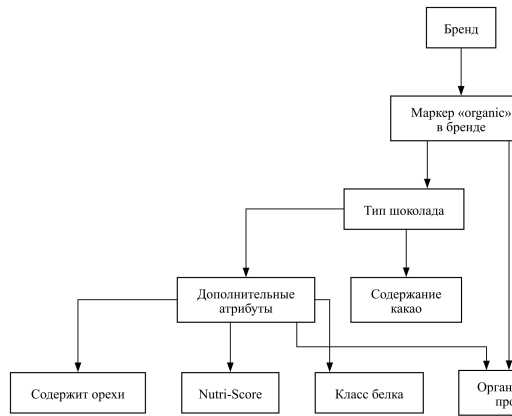


В2. Байесовский слой: роль валидатора плотностей вместо классификатора

Что построено. Байесовская сеть на структуре Hill Climb + BIC (на серебряной разметке, 15 тыс. товаров на непересекающихся товарных кодах (code-disjoint)); условные вероятности доучены на `CONCAT(silver, gold-train ×10)` с априорной BDeu.

Роль классификатора отвергнута. Эмпирическая точность на трудном остатке после слоя 2 — около 52 %, ниже мажорной точки отсчёта. Сеть переориентирована в валидатор плотностей: $P(\hat{y} \mid \text{evidence})$ ниже 5-го перцентиля — ячейка помечается и понижается на следующий слой.

Производственное включение — точечное. Из 20 пар «категория-атрибут» валидатор активирован только на одной паре: chocolate / contains_nuts



В3. Серебряный сигнал vs контрольная разметка

Серебряная разметка (консенсус тегов OFF + 3 языковые модели + слепая разметка Orpus) служит **обучающим** сигналом. Контрольная выборка получена по аудит-циклу.

Цикл «аудит — правки экстрактора — переобучение — измерение» воспроизведён для всех 3 категорий с приростом **+18,2, +16,3, +14,4** пп.

Калибровка уверенности — комбинация Platt + изотонической регрессии поатрибутно; ошибка ECE измеряется на отдельной выборке.

В4. Утечки и борьба с ними

Запрет утечки `categories_tags` в эмбединги.

Эмбединги Слой 2 строятся **только** из партнёрских полей: `product_name + brands + ingredients_text + quantity.categories_tags` — это **выход** обогащения, не вход; использование как признака привело бы к утечке целевой переменной.

Разбиение по непересекающимся товарным кодам (60/20/20; brand overlap train↔eval 82–85%). Бренды в обучающей, валидационной и контрольной выборках не пересекаются — гарантия обобщения на новые бренды.

Анализ по расстоянию k-NN (§3.3.7.3): точность **не падает** на удалённых от обучающей выборки товарах — монотонный рост от Q1 (82,7 %) до Q4 (91,6 %).

B5. Многоязычность

Языковой состав OFF: $\approx 40\%$ FR, 10% EN, 7% DE, остальное — прочие.

Эмбединг: `paraphrase-multilingual-MiniLM-L12-v2` (50+ языков, общее семантическое пространство).

Поатрибутно-поязыковой срез ($n=3257$ ячеек): взвешенная точность стабильна по основным языкам.

Под балансировку каскад не оптимизировался — это естественный выигрыш многоязычной базовой модели эмбедингов.

В6. Стоимость языковой модели на 1 млн SKU

Каскад делегирует языковой модели лишь 7,1 % ячеек ($n = 3506$) — прямая прокси к стоимости. Сокращение затрат **92,9 % (в 14× раз)** относительно «прямой языковой модели на всё».

Распределение по слоям: L1 (regex высокой точности) — 18,4 %, L2 (ML) — **73,8 %**, L3 (regex низкой точности) — 0,7 %, L4 (языковая модель) — 7,1 %.

Проекция на 1 млн SKU × 22 атрибута ($\sim 2,2 \cdot 10^7$ ячеек):

- ▶ Прямая модель — оплачены все $\sim 2,2 \cdot 10^7$ вызовов.
- ▶ Каскад — оплачены лишь $\sim 1,6 \cdot 10^6$ вызовов (7,1 %).

Топ-3 затратных атрибутов для языковой модели: pasta.grain_type (32,9 %), pasta.pasta_shape (22,8 %), cheeses.is_pdo (13,6 %).

В7. Холодный старт: электроника (§5 ноутбука)

Демонстрация переноса методики на **новую категорию без регулярных правил**.

Электроника (8 атрибутов): смартфоны — RAM, объём памяти, диагональ экрана, поддержка 5G, ...

Перенос: активны только Слой 2 (ML + эмбединги) и Слой 4 (языковая модель); Слой 1 пропускается; Слой 3 — по обучающим парам.

Результат: холодный старт работает на ограниченной обучающей выборке, что подтверждает применимость каскада за пределами продуктового домена.